INSTITUTO TECNOLÓGICO SUPERIOR DEL ESTADO DE HIDALGO

INGENIERÍA EN SISTEMAS COMPUTACIONALES

Métodos Numéricos

Evidencia Problemario

Dr. Efrén Rolando Romero León

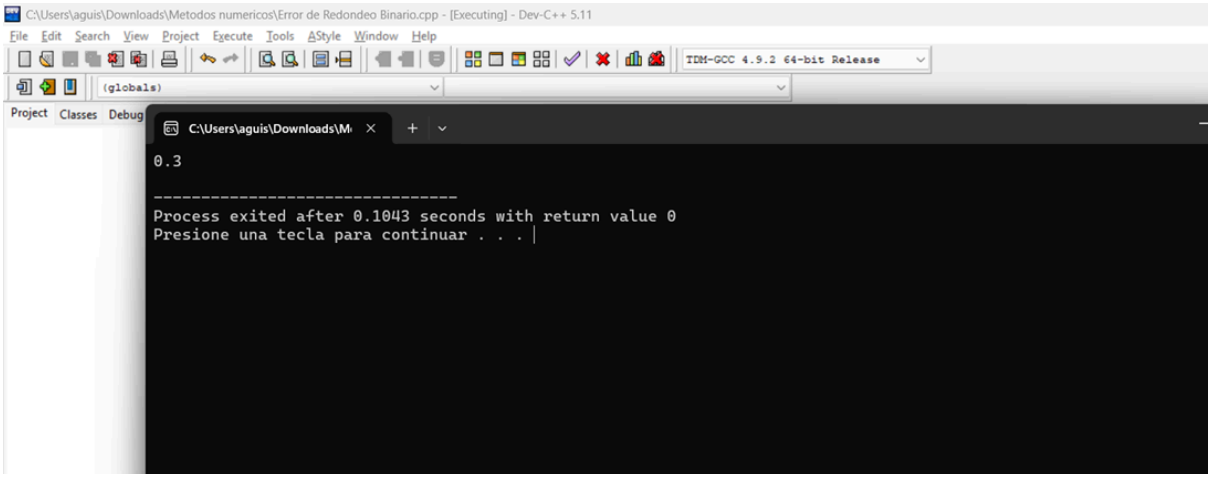
Aguilar Sanchez Ariana
Arellano Rivera Fatima Yoilyn
Rodríguez Gómez Gerardo Imanol
Montalvo Vigil Leonel

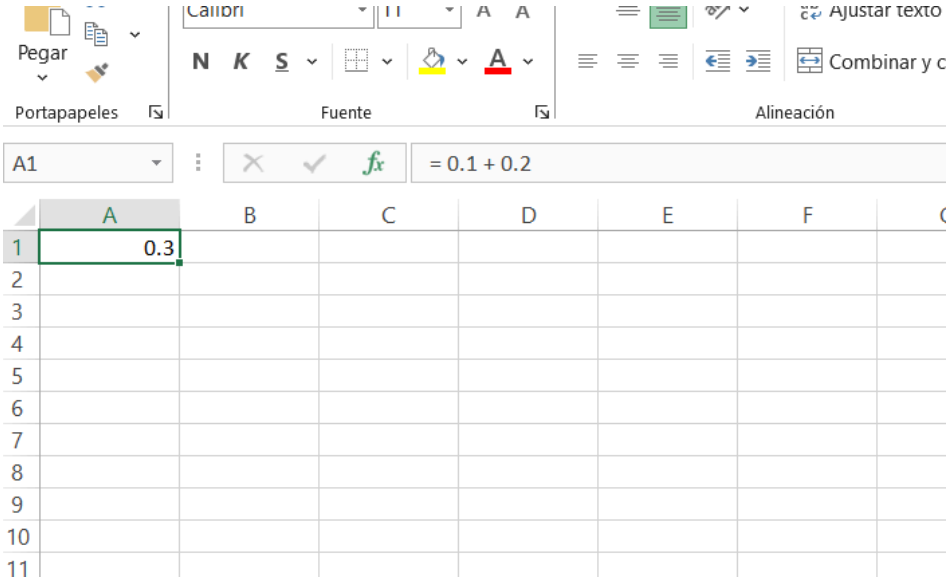
Tabla de contenido

1. Error de Redondeo Binario..... 3
2. Pérdida de Precisión por Magnitud(IEEE 754)..... 4
3.Comparación Directa con ==..... 7
4. Acumulación de Errores en Bucles.....9
5.Cancelación por Resta (Loss of Significance)..... 12
6.Conversión Estrecha (Narrowing Primitive Conversión)..... 14
7. Desbordamiento (Overflow) en Punto Flotante.....16
8. NaN (Not a Number)..... 18
Gráfico Gantt..... 20
Tablero Scrum..... 21
Evidencia de trabajo en equipo..... 22
Referencias Bibliográficas..... 23
Conclusiones.....23

1. Error de Redondeo Binario

```
#include <iostream>

using namespace std;

int main() {

    cout << 0.1 + 0.2 << endl;

    return 0;

}
```

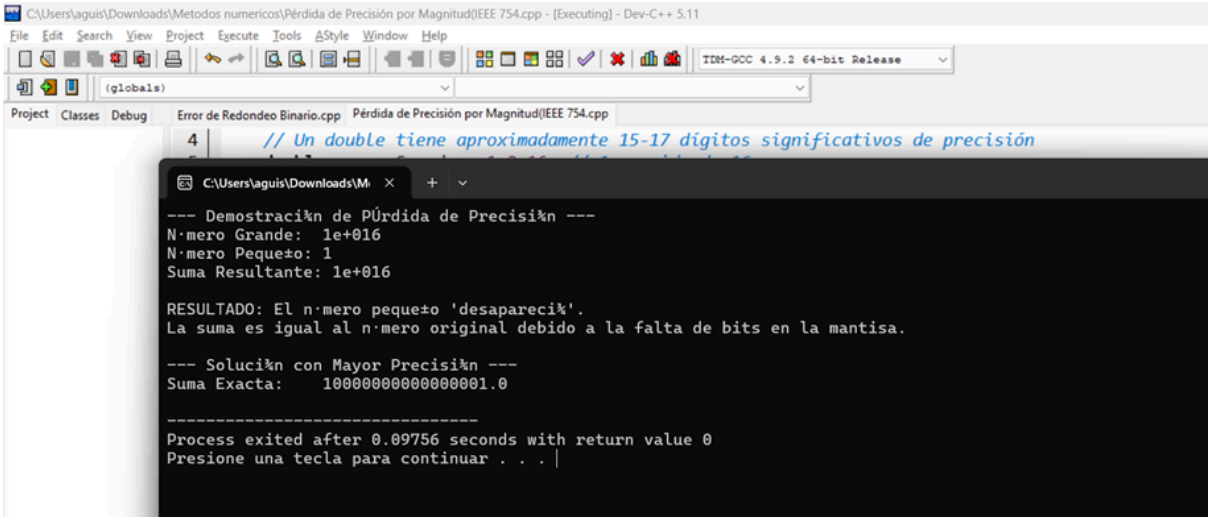




2. Pérdida de Precisión por Magnitud(IEEE 754)

```
#include <iostream>

#include <iomanip> // Necesario para el formato de salida

int main() {

    // Un double tiene aproximadamente 15-17 dígitos significativos de precisión

    double numeroGrande = 1.0e16; // 1 seguido de 16 ceros

    double numeroPequeno = 1.0;

    double resultado = numeroGrande + numeroPequeno;


    std::cout << "--- Demostración de Pérdida de Precisión ---" << std::endl;

    std::cout << "Número Grande:  " << numeroGrande << std::endl;

    std::cout << "Número Pequeño: " << numeroPequeno << std::endl;

    std::cout << "Suma Resultante: " << resultado << std::endl;
```

```
    // Verificación lógica
    if (resultado == numeroGrande) {
    std::cout << "\nRESULTADO: El número pequeño 'desapareció'." <<std::endl;
    std::cout << "La suma es igual al número original debido a la falta de
    bits en la mantisa." << std::endl;
    }

    // --- SOLUCIÓN (Equivalente a BigDecimal en C++ Estándar) ---
    /* En C++ estándar usamos 'long double' para mayor precisión.
       Esto corrige el error de este ejemplo específico.
    */
    long double bdGrande = 1.0e16L; // La 'L' fuerza a que sea long double
    long double bdPequeno = 1.0L;
    long double bdResultado = bdGrande + bdPequeno;

    std::cout << "\n--- Solución con Mayor Precisión ---" << std::endl;

    // Configuración para simular .toPlainString() y ver los decimales
    std::cout << std::fixed << std::setprecision(1);

    std::cout << "Suma Exacta:    " << bdResultado << std::endl;

    return 0;
}
```


C:\Users\aguis\Downloads\Metodos numericos\Pérdida de Precisión por Magnitud(IEEE 754.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
Error de Redondeo Binario.cpp
Pérdida de Precisión por Magnitud(IEEE 754.cpp
4 // Un double tiene aproximadamente 15-17 dígitos significativos de precisión
C:\Users\aguis\Downloads\M
--- Demostraci%n de PÚrdida de Precisi%n ---
N·mero Grande: 1e+016
N·mero Peque±o: 1
Suma Resultante: 1e+016
RESULTADO: El n·mero peque±o 'desapareci%'.
La suma es igual al n·mero original debido a la falta de bits en la mantisa.
--- Soluci%n con Mayor Precisi%n ---
Suma Exacta: 10000000000000001.0

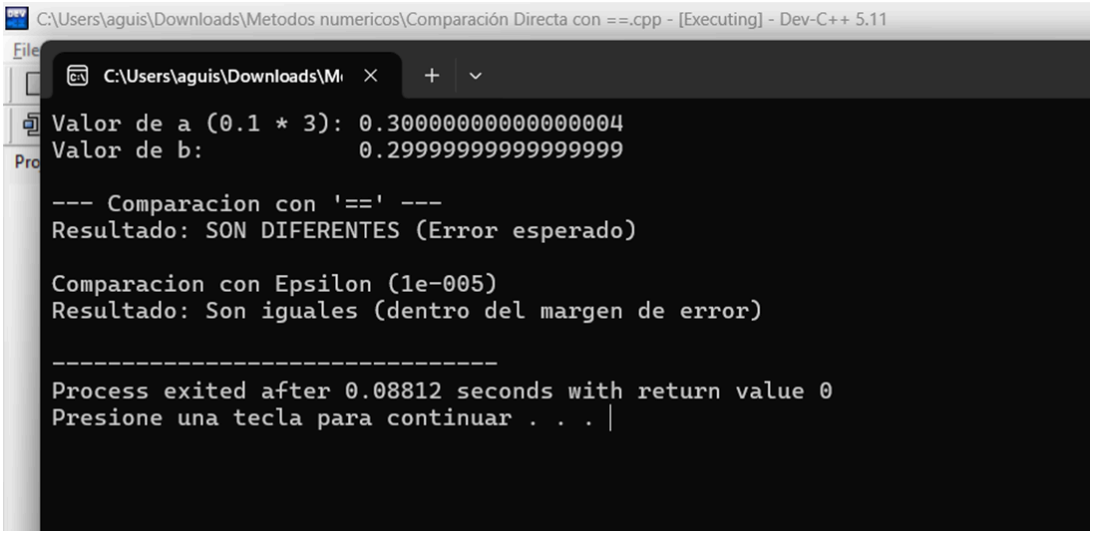
Process exited after 0.09756 seconds with return value 0
Presione una tecla para continuar . . .

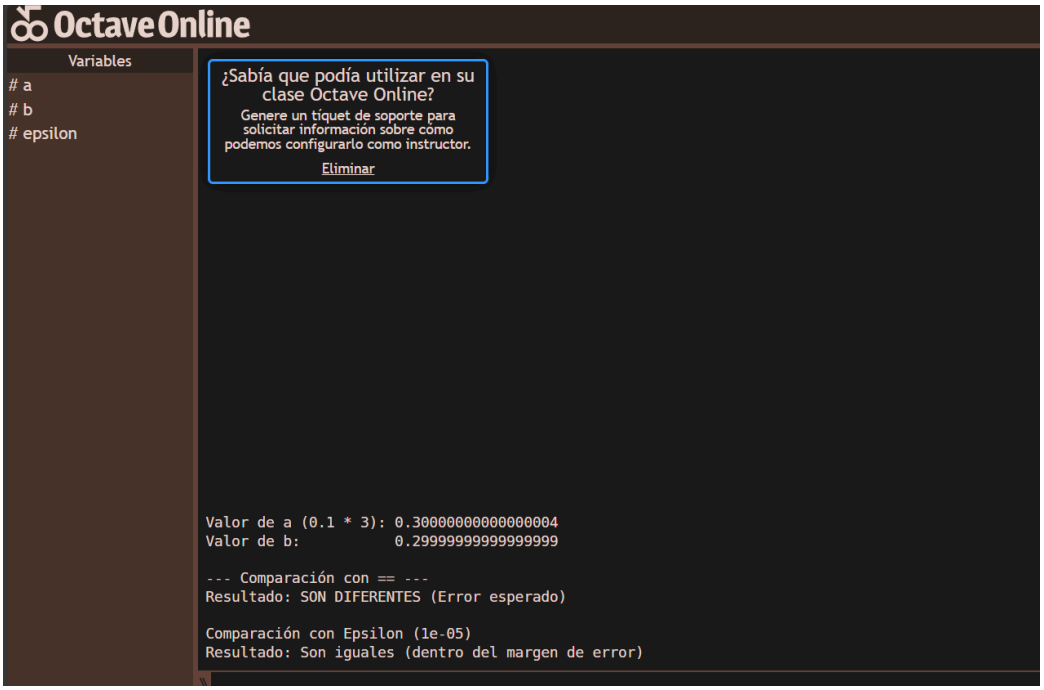
Octave Online
Variables
numeroGrande
numeroPequeno
resultado
¿Sabía que podía utilizar en su clase Octave Online?
Genere un tíquet de soporte para solicitar información sobre cómo podemos configurarlo como instructor.
Eliminar
--- Demostración de Pérdida de Precisión ---
Número Grande: 10000000000000000
Número Pequeño: 1
Suma Resultante: 10000000000000000
RESULTADO: El número pequeño "desapareció".
La suma es igual al número original por pérdida de precisión.

3.Comparación Directa con ==

```
#include <iostream>

#include <cmath>   // Necesario para std::abs

#include <iomanip> // Necesario para std::setprecision


int main() {

    // 1. El Problema: Error de redondeo en aritmética de punto flotante

    double a = 0.1 + 0.1 + 0.1;

    double b = 0.3;


    // Configuración para mostrar todos los decimales y ver el error

    std::cout << std::setprecision(17);


    std::cout << "Valor de a (0.1 * 3): " << a << std::endl;

    std::cout << "Valor de b:           " << b << std::endl;


    // 2. Intento de comparación directa (FALLARÁ)

    std::cout << "\n--- Comparacion con '==' ---" << std::endl;


    if (a == b) {

        std::cout << "Resultado: Son iguales" << std::endl;

    } else {

        std::cout <<"Resultado: SON DIFERENTES (Error esperado)"<< std::endl;

    }

    // 3. La Solución: Uso de un margen de error (Épsilon)

    double epsilon = 0.00001;
```

```
    // --- LÍNEA CORREGIDA AQUÍ ---
    // Quitamos 'std::defaultfloat' para compatibilidad con compiladores viejos
    std::cout << std::setprecision(6);
    std::cout <<"\nComparacion con Epsilon (" << epsilon << ")"<< std::endl;
    if (std::abs(a - b) < epsilon) {
        std::cout << "Resultado: Son iguales (dentro del margen de error)" << std::endl;
    } else {
        std::cout << "Resultado: Son diferentes" << std::endl;
    }
    return 0;
}
```

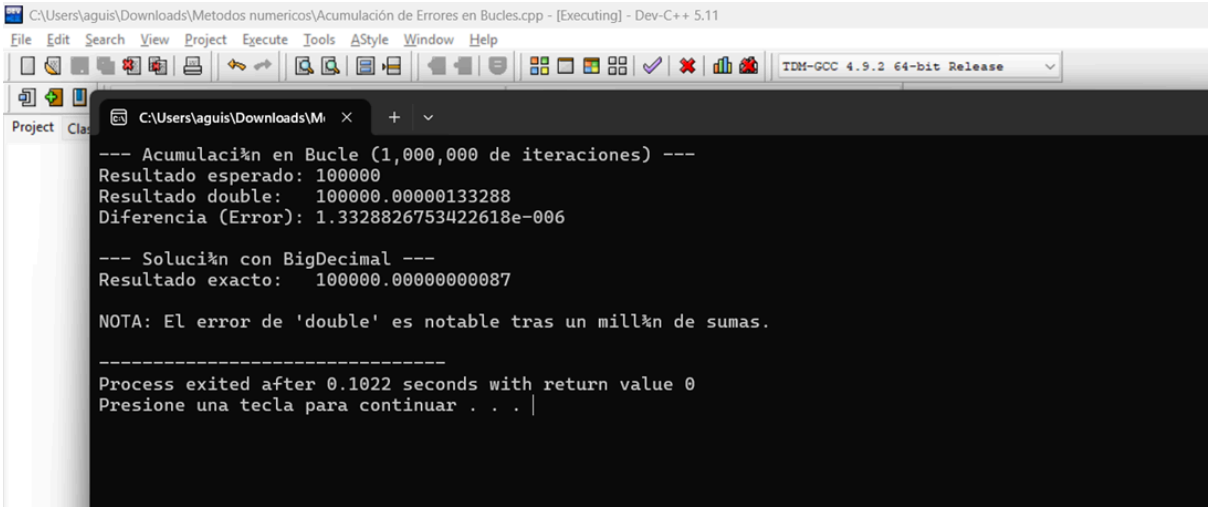




4. Acumulación de Errores en Bucles

```
#include <iostream>

#include <iomanip> // Librería necesaria para controlar la impresión de números

int main() {

    int iteraciones = 1000000; // Un millón de sumas

    double incremento = 0.1;

    // 1. Acumulación usando 'double'

    double sumaDouble = 0.0;

    for (int i = 0; i < iteraciones; i++) {

        sumaDouble += incremento;

    }

    // El resultado esperado debería ser exactamente 100,000.0

    double esperado = iteraciones * incremento;


    std::cout << "--- Acumulación en Bucle (1,000,000 de iteraciones) ---" << std::endl;


    // Configuración importante:

    // Le decimos a C++ que imprima con máxima precisión (17 dígitos)

    // para que se vea el error y no lo redondee visualmente.

    std::cout.precision(17);


    std::cout << "Resultado esperado: " << esperado << std::endl;

    std::cout << "Resultado double:   " << sumaDouble << std::endl;


    // Aquí verás el error de 1.3328...e-05 (o similar)

    std::cout << "Diferencia (Error): " << (sumaDouble - esperado) << std::endl;
```

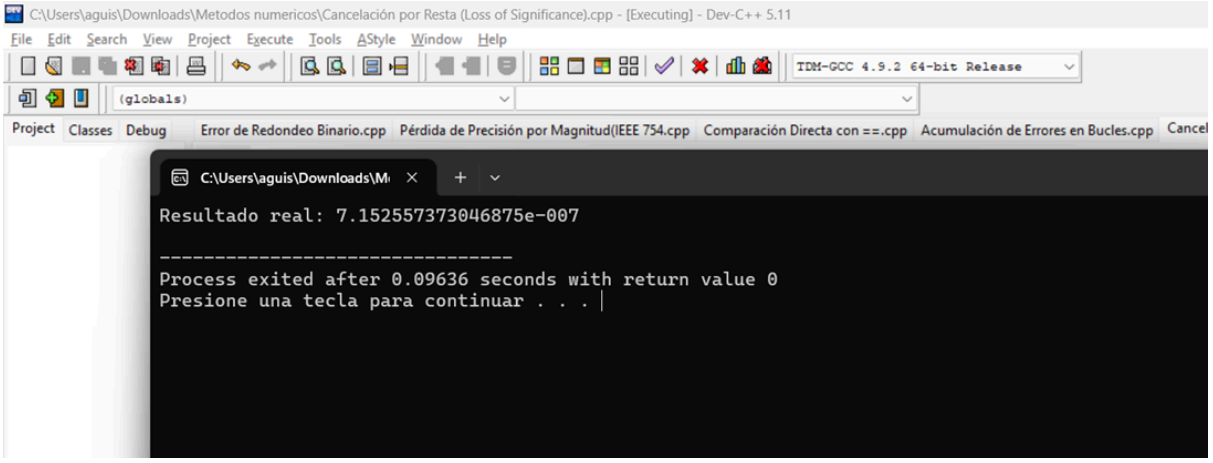
```
    // 2. Solución usando 'BigDecimal' (Simulado con long double en C++)
    // Usamos 'long double' para que esta parte sí de exacto, simulando el BigDecimal
    long double sumaBD = 0.0L;
    long double incrementoBD = 0.1L;

    for (int i = 0; i < iteraciones; i++) {
        sumaBD = sumaBD + incrementoBD;
    }

    std::cout << "\n--- Solución con BigDecimal ---" << std::endl;
    std::cout << "Resultado exacto:   " << sumaBD << std::endl;

    // 3. Verificación de error
    if (sumaDouble != esperado) {
        std::cout << "\nNOTA: El error de 'double' es notable tras un millón de sumas." << std::endl;
    }

    return 0;
}
```



Inicio
SUBES - 2024
SUBES - 2024
Y2Mate - Youtube...
YTMP3 - YouTube...
YTMP3 - YouTube...
Octave Online
Variables
esperado
i
incremento
iteraciones
sumaDouble
¿Sabía que podía utilizar en su clase Octave Online?
Genere un tíquet de soporte para solicitar información sobre cómo podemos configurarlo como instructor.
Eliminar
--- Acumulación en Bucle (1,000,000 de iteraciones) ---
Resultado esperado: 100000.00000000000000000
Resultado double: 100000.00000133288267534
Diferencia (Error): 1.33288267534226179e-06
NOTA: El error de punto flotante es notable tras un millón de sumas.

5.Cancelación por Resta (Loss of Significance)

```
#include <iostream>

#include <iomanip> // Necesario para manipular la salida


int main() {

    // Dos números muy grandes y muy cercanos

    double x = 1234567890.1234567;

    double y = 1234567890.1234560;


    // El resultado esperado es 0.0000007

    double resultado = x - y;


    // Configuración para ver TODOS los decimales y notar el error/basura

    std::cout.precision(20);


    std::cout << "Resultado real: " << resultado << std::endl;

    // En 2026 notarás que el resultado es 0.0 o un valor basura

    // debido a que los últimos dígitos se perdieron al almacenar x e y.


    return 0;

}
```

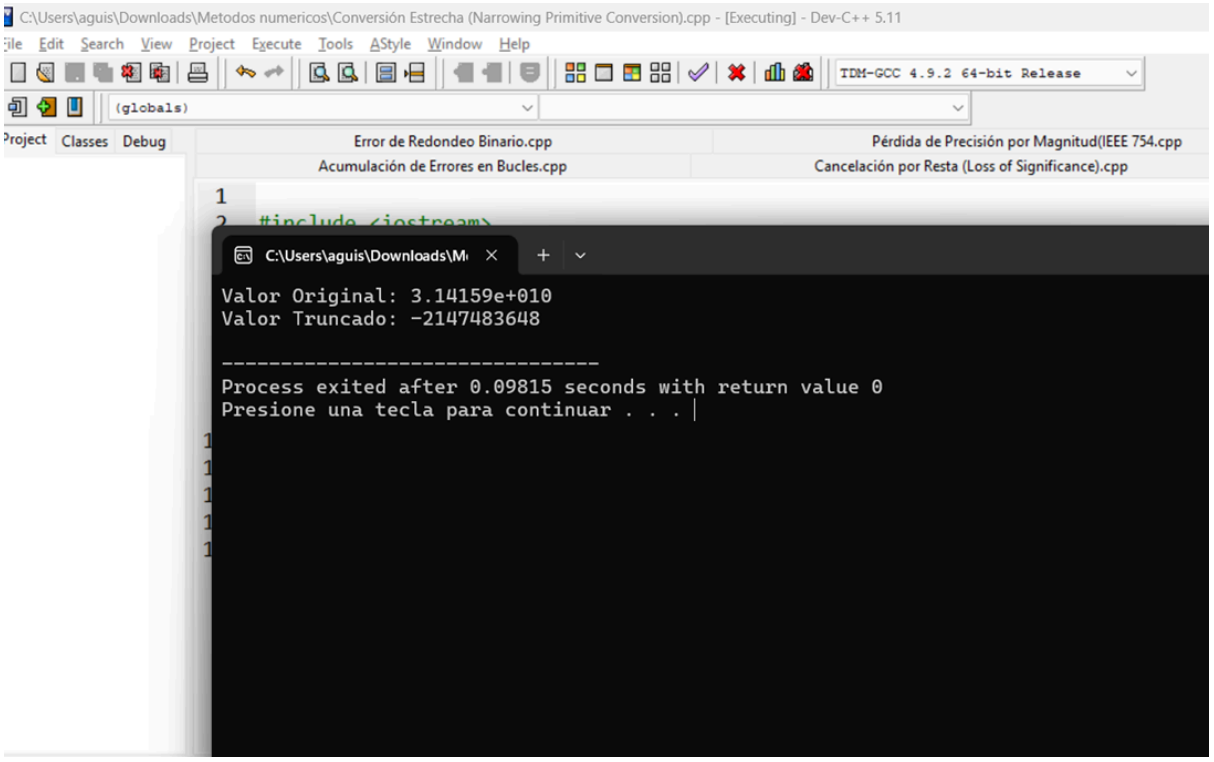

C:\Users\aguis\Downloads\Metodos numericos\Cancelación por Resta (Loss of Significance).cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
Error de Redondeo Binario.cpp
Pérdida de Precisión por Magnitud(IEEE 754.cpp
Comparación Directa con ==.cpp
Acumulación de Errores en Bucles.cpp
Resultado real: 7.152557373046875e-007

Process exited after 0.09636 seconds with return value 0
Presione una tecla para continuar . . .

Octave Online
Variables
resultado
x
y
¿Sabía que podía utilizar en su clase Octave Online?
Genere un tíquet de soporte para solicitar información sobre cómo podemos configurarlo como instructor.
Eliminar
x = 1234567890.12345671653747559
y = 1234567890.12345600128173828
Resultado de x - y:
0.00000071525573730469

6.Conversión Estrecha (Narrowing Primitive Conversión)

```
#include <iostream>

int main() {

    double valorDouble = 3.14159e10; // Un número muy grande

    int valorInt = (int) valorDouble; // Casting explícito


    std::cout << "Valor Original: " << valorDouble << std::endl;

    std::cout << "Valor Truncado: " << valorInt << std::endl;

    // Imprime 2147483647 (el máximo int) porque el double no cabe.


    return 0;

}
```



Octave Online

Variables

ans

valorDouble

? valorInt

¿Sabía que podía utilizar en su clase Octave Online?

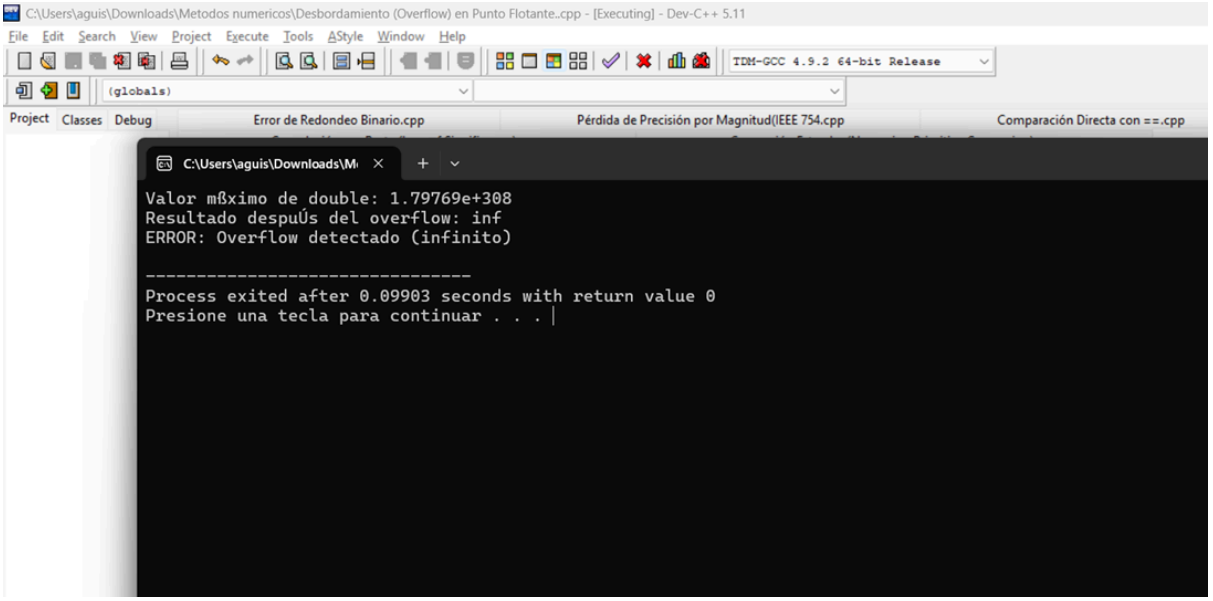
Genere un tíquet de soporte para solicitar información sobre cómo podemos configurarlo como instructor.

Eliminar

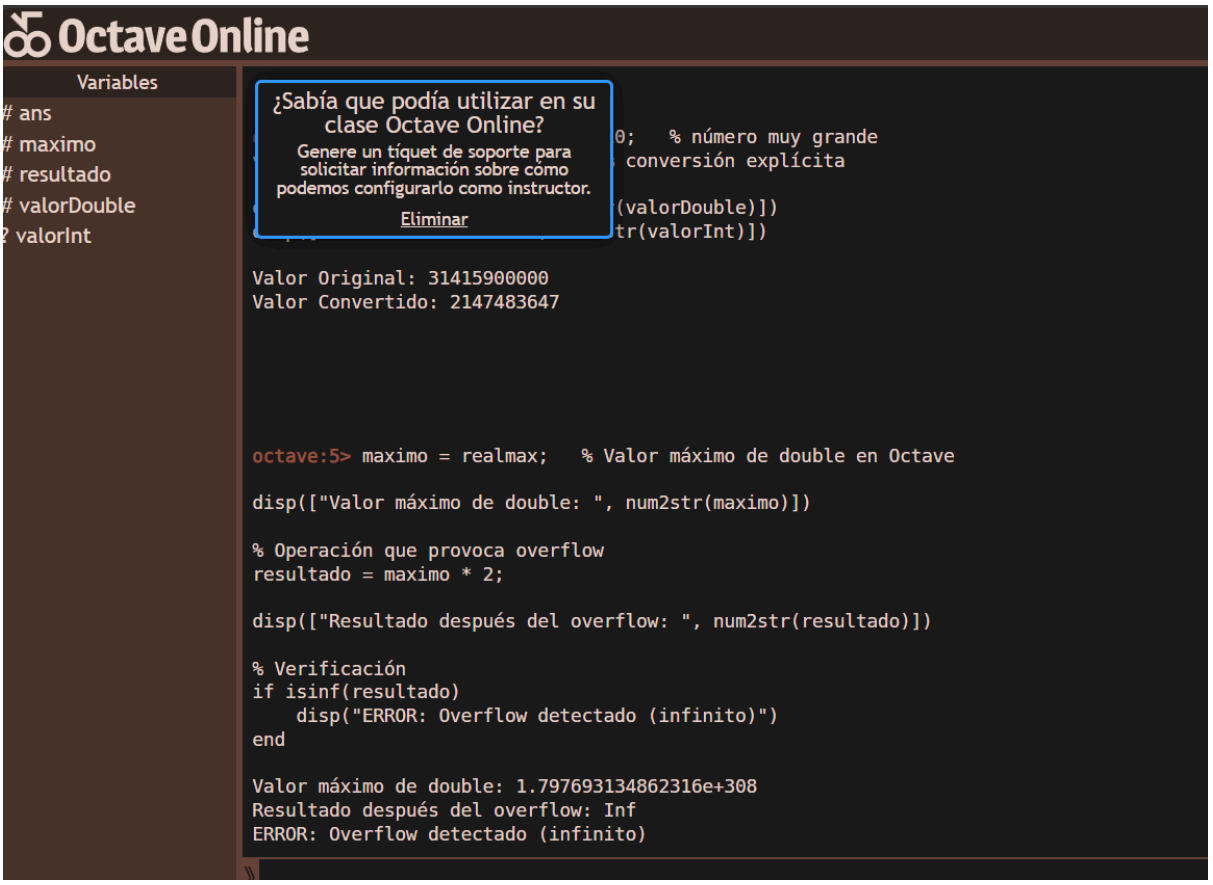
```
octave:1> valorDouble = 3.14159e10;   % número muy grande
valorInt = int32(valorDouble);  % conversión explícita

disp(["Valor Original: ", num2str(valorDouble)])
disp(["Valor Convertido: ", num2str(valorInt)])

Valor Original: 31415900000
Valor Convertido: 2147483647
```

7. Desbordamiento (Overflow) en Punto Flotante

```
 #include <iostream>

#include <limits> // Para valores máximos del tipo double


int main() {

    double maximo = std::numeric_limits<double>::max();


    std::cout << "Valor máximo de double: " << maximo << std::endl;


    // Operación que provoca overflow

    double resultado = maximo * 2;


    std::cout << "Resultado después del overflow: " << resultado << std::endl;


    // Verificación

    if (resultado == std::numeric_limits<double>::infinity()) {

        std::cout << "ERROR: Overflow detectado (infinito)" << std::endl;

    }


    return 0;

}
```

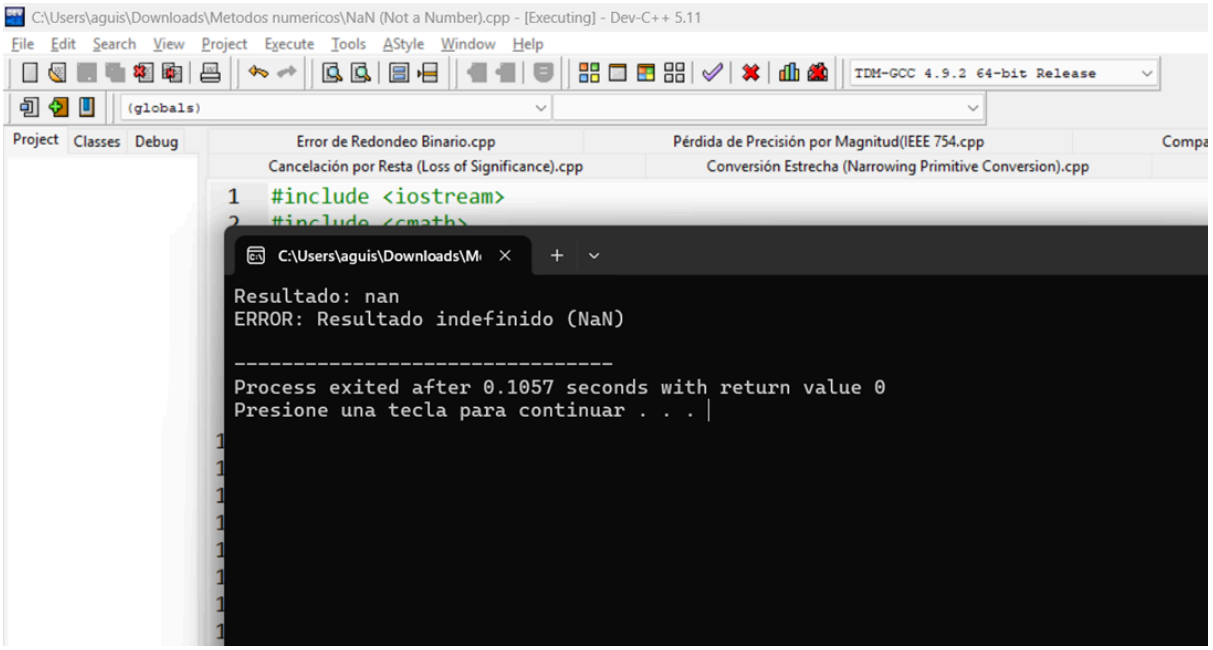

C:\Users\aguis\Downloads\Metodos numericos\Desbordamiento (Overflow) en Punto Flotante..cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
Error de Redondeo Binario.cpp
Pérdida de Precisión por Magnitud(IEEE 754.cpp
Comparación Directa con ==.cpp
C:\Users\aguis\Downloads\M
Valor mßximo de double: 1.79769e+308
Resultado despuÚs del overflow: inf
ERROR: Overflow detectado (infinito)

Process exited after 0.09903 seconds with return value 0
Presione una tecla para continuar . . .


Octave Online
Variables
ans
maximo
resultado
valorDouble
valorInt
¿Sabía que podía utilizar en su clase Octave Online?
Genere un tíquet de soporte para solicitar información sobre cómo podemos configurarlo como instructor.
Eliminar
0; % número muy grande
conversión explícita
(valorDouble)])
tr(valorInt)])
Valor Original: 31415900000
Valor Convertido: 2147483647
octave:5> maximo = realmax; % Valor máximo de double en Octave
disp(["Valor máximo de double: ", num2str(maximo)])
% Operación que provoca overflow
resultado = maximo * 2;
disp(["Resultado después del overflow: ", num2str(resultado)])
% Verificación
if isinf(resultado)
disp("ERROR: Overflow detectado (infinito)")
end
Valor máximo de double: 1.797693134862316e+308
Resultado después del overflow: Inf
ERROR: Overflow detectado (infinito)

8. NaN (Not a Number)

```
#include <iostream>

#include <cmath>


int main() {

   double a = 0.0;

   double b = 0.0;

    double resultado = a / b; // 0 / 0 ? NaN

    std::cout << "Resultado: " << resultado << std::endl;

    if (std::isnan(resultado)) {

      std::cout << "ERROR: Resultado indefinido (NaN)" << std::endl;

   }

    return 0;

}
```



Octave Online
Variables
a
ans
b
resultado
¿Sabía que podía utilizar en su clase Octave Online?
Genere un tíquet de soporte para solicitar información sobre cómo podemos configurarlo como instructor.
Eliminar
octave:1> a = 0.0;
b = 0.0;
resultado = a / b; % 0 / 0 → NaN
disp(["Resultado: ", num2str(resultado)])
if isnan(resultado)
    disp("ERROR: Resultado indefinido (NaN)")
end
Resultado: NaN
ERROR: Resultado indefinido (NaN)

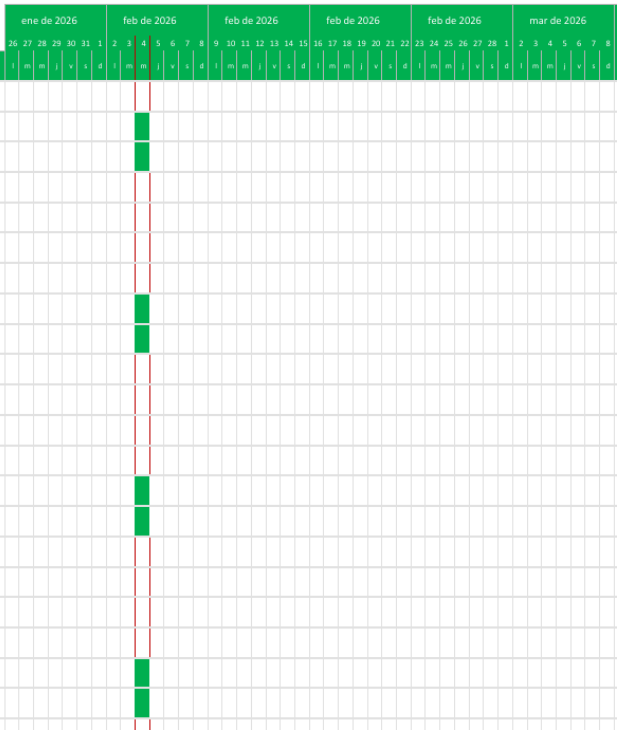
Gráfico Gantt

T1E2Problemario

GRÁFICO GANTT

Inicio del proyecto: mié, 04/02/2026

TAREA	ASIGNADO A	PROGRESO	INICIO	FIN
Configuración				
Organización y Roles (Tablero Sc	Imanol y Fatima	100%	4-2-26	4-2-26
Configuración de Groupware (En	Leonel, Imanol Fatima	100%	4-2-26	4-2-26
Desarrollo				
Resolución de Ejercicios Chapra I	Ariana	100%	4-2-26	4-2-26
Cálculos Matemáticos de Aproxi	Leonel	100%	4-2-26	4-2-26
Validacion				
Comprobación en Software	Ariana	100%	4-2-26	4-2-26
Pruebas de Usabilidad (UX/UI) d	Leonel , imanol	100%	4-2-26	4-2-26
Cierre				
Redacción del Reporte Final	Todos	100%	4-2-26	4-2-26
Integración de Evidencias y Ane	Todos	100%	4-2-26	4-2-26

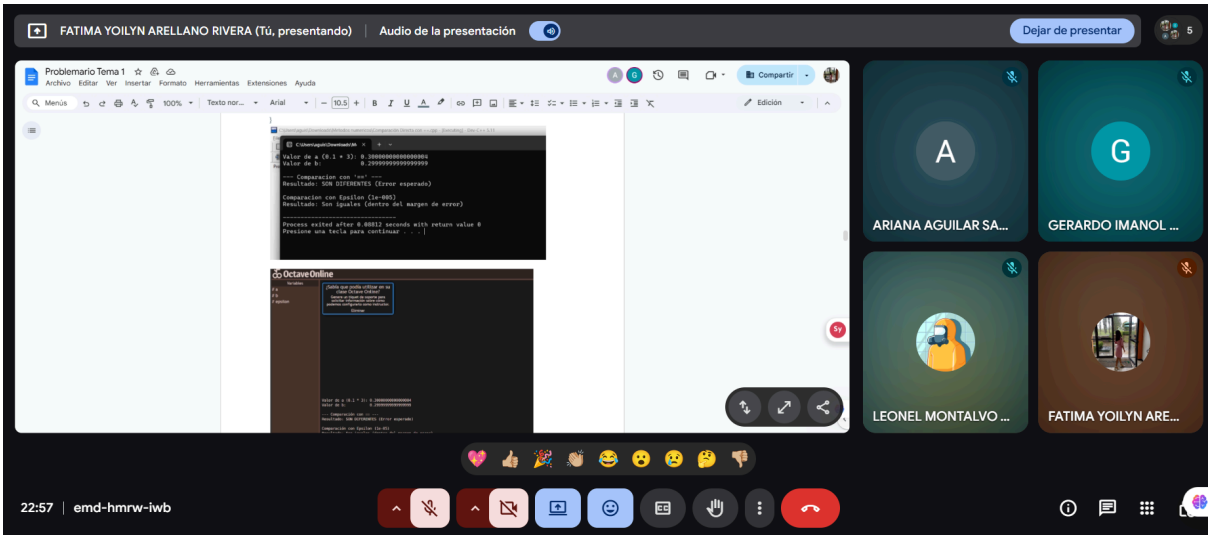


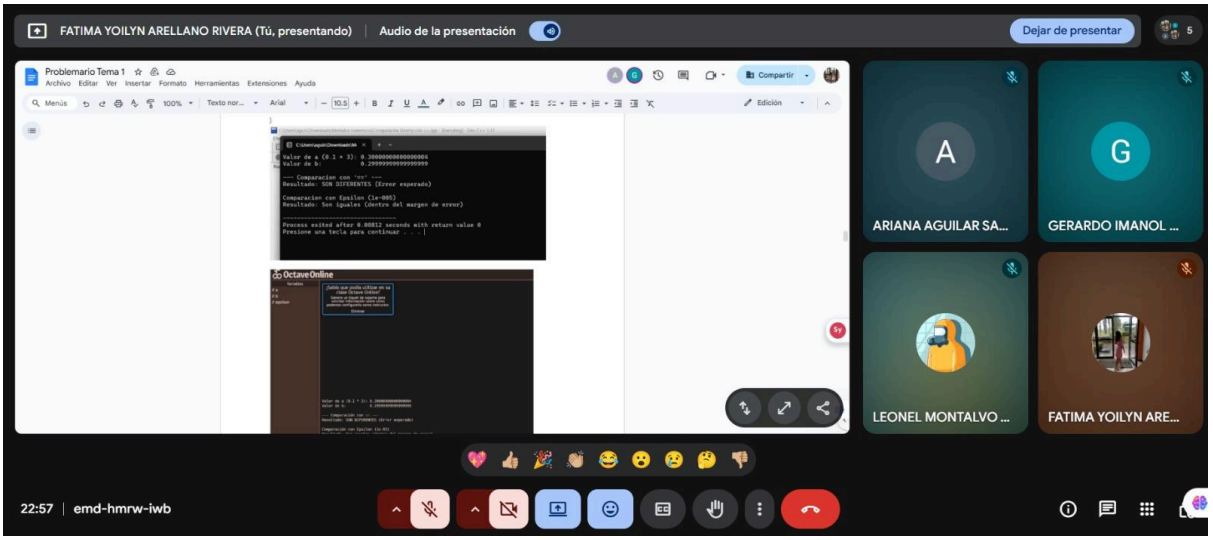
Tablero Scrum

Leyenda				
● Ariana Aguilar ● Fátima Arellano ● Gerardo Rodríguez ● Leonel Vigil				
Atrasos	**Qué hacer**	**En curso**	**Consulte**	**Hecho**
	Investigar más sobre IEEE 754	Revisar formato final del documento	Validar ejemplos con el profesor	Error de Redondeo Binario
		Preparar presentación del proyecto	Revisar comentarios del código	Pérdida de Precisión (IEEE 754)
				Comparación con ==
				Acumulación de Errores
				Cancelación por Resta
				Conversión Estrecha
				Desbordamiento (Overflow)
				NaN (Not a Number)

	A	B	C	D	E	F
1	SPRINT BACKLOG - PROBLEMARIO MÉTODOS NUMÉRICOS					
2						
3	#	Tarea	Responsable	Estado	Prioridad	Horas
4	1	Error de Redondeo Binario	Ariana	Hecho	Alta	2
5	2	Pérdida de Precisión por Magnitud	Fátima	Hecho	Alta	3
6	3	Comparación Directa con ==	Gerardo	Hecho	Alta	2
7	4	Acumulación de Errores en Bucles	Leonel	Hecho	Alta	3
8	5	Cancelación por Resta	Ariana	Hecho	Media	2
9	6	Conversión Estrecha (Narrowing)	Fátima	Hecho	Media	2
10	7	Desbordamiento (Overflow)	Gerardo	Hecho	Alta	2
11	8	NaN (Not a Number)	Leonel	Hecho	Media	1
12	9	Revisar formato del documento	Ariana	En curso	Alta	2
13	10	Preparar presentación	Fátima	En curso	Media	3
14	11	Validar ejemplos con profesor	Leonel	Consulte	Alta	2
15	12	Revisar comentarios del código	Ariana	Consulte	Media	1
16	13	Investigar más sobre IEEE 754	Gerardo	Qué hacer	Baja	2
17						
18						

Evidencia de trabajo en equipo





Referencias Bibliográficas

Aprende con Alf. (2023, 18 de marzo). *Tipos de datos en C++: Float y double*. https://aprendeconalf.es/docencia/c/tipos-datos/

Código Facilito. (2024, 5 de junio). *Errores de precisión en punto flotante* [Video]. YouTube. https://www.youtube.com/codigofacilito

Departamento de Informática, Universidad de Valladolid. (2022). *Representación de números en punto flotante*. http://www.infor.uva.es/~cevp/MTI/representacion_numeros.pdf

Facultad de Ingeniería, UNAM. (2023). *Métodos numéricos: Errores de redondeo y truncamiento*. http://www.ingenieria.unam.mx/~metodosnumericos/errores.html

Lenguaje C. (2024, 10 de enero). *Números de punto flotante en C y C++*. https://www.lenguajec.com/tipos-de-datos-punto-flotante/

OpenWebinars. (2023, 22 de septiembre). *Aritmética de punto flotante: Errores comunes*. https://openwebinars.net/blog/aritmetica-punto-flotante/

Pérez, J. C. (2022). *El estándar IEEE 754 para representación de números reales*. Revista Española de Informática Aplicada. https://www.reia.es/articulos/ieee754-estandar

Programación en C++. (2024). *Conversión de tipos de datos (Type casting)*. https://programacioncpp.com/conversion-tipos-datos/

Programación Fácil. (2023, 14 de noviembre). *¿Por qué 0.1 + 0.2 no es igual a 0.3?* [Entrada de blog]. https://programacionfacil.com/blog/punto-flotante-precision/

Universidad Autónoma de Madrid. (2023). *Introducción a los métodos numéricos*. Departamento de Matemáticas. https://www.uam.es/matematicas/metodos-numericos/errores

Universidad Politécnica de Madrid. (2022). *Análisis numérico: Propagación de errores*. Escuela Técnica Superior de Ingenieros Informáticos. https://www.fi.upm.es/docs/analisis-numerico/propagacion-errores.pdf

Velázquez, M. A. (2023, 8 de abril). *Overflow y underflow en aritmética computacional*. Tecnología Informática Educativa. https://www.tecnologiaeducativa.es/overflow-underflow/

Conclusiones

Aguilar Sanchez Ariana
Como conclusión después de analizar cada código que los errores en métodos numéricos no sólo provienen de una aproximación de cálculos si no también a la representación numérica de las computadoras, estos ejemplos resaltan la importancia de que validemos los datos de entrada.

Arellano Rivera Fatima Yoilyn
En resumen, el análisis de estos códigos demuestra que los errores en métodos numéricos surgen no solo de aproximaciones en los cálculos, sino también de la representación limitada de los números en las computadoras. Este trabajo fue importante para el entendimiento de cómo validar entradas y controlar estos errores para garantizar resultados precisos y confiables.

Montalvo Vigil Leonel
Con la realización de este trabajo fue posible reforzar los conceptos relacionados con los distintos tipos de errores en los métodos numéricos. A través del desarrollo y resolución de los ejercicios, se logró comprender cómo estos errores influyen directamente en los resultados de los cálculos numéricos y en la confiabilidad de las aproximaciones obtenidas.

Rodriguez Gomez Gerardo Imanol
Este trabajo me sirvió para ver que los errores numéricos pasan más seguido de lo que pensaba. Los números no siempre se guardan exactos y por eso a veces los resultados fallan. La verdad si fue algo pesado, pero al menos quedó un poco mas claro como funciona eso.